Embedded Control for Robotics

Lecture # 09: **Analog to Digital Conversion**

- Book # 1: Chapter 5 : Embedded Digital Control with Microcontrollers - Implementation with C and Python

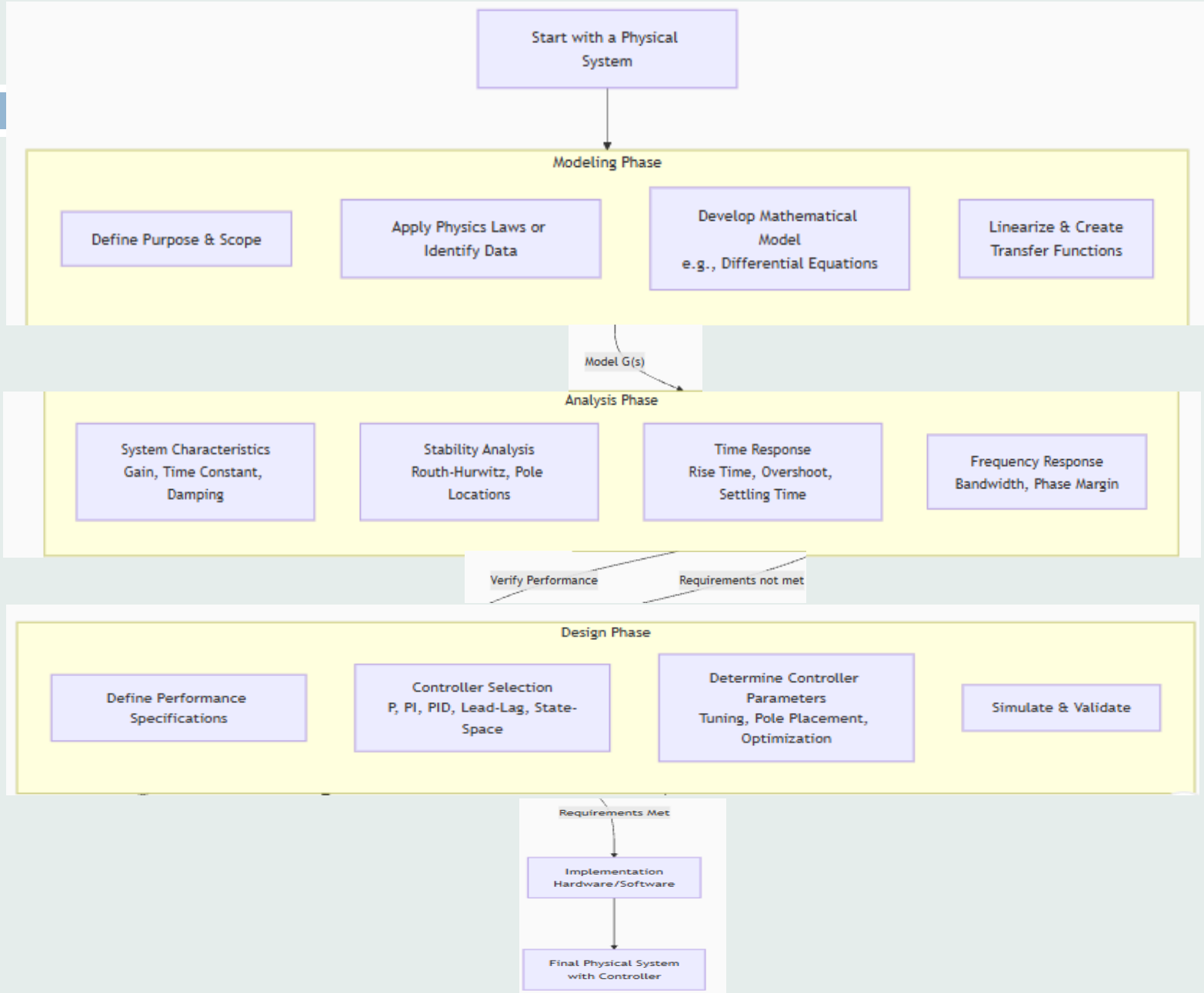
Dr. Suleman Khan
CS, NUCES-FAST, Islamabad

Overview Today's Lecture

- Review
- Why ADC/DAC?
- ADC
- Examples
- DAC

Control Systems: Big Picture

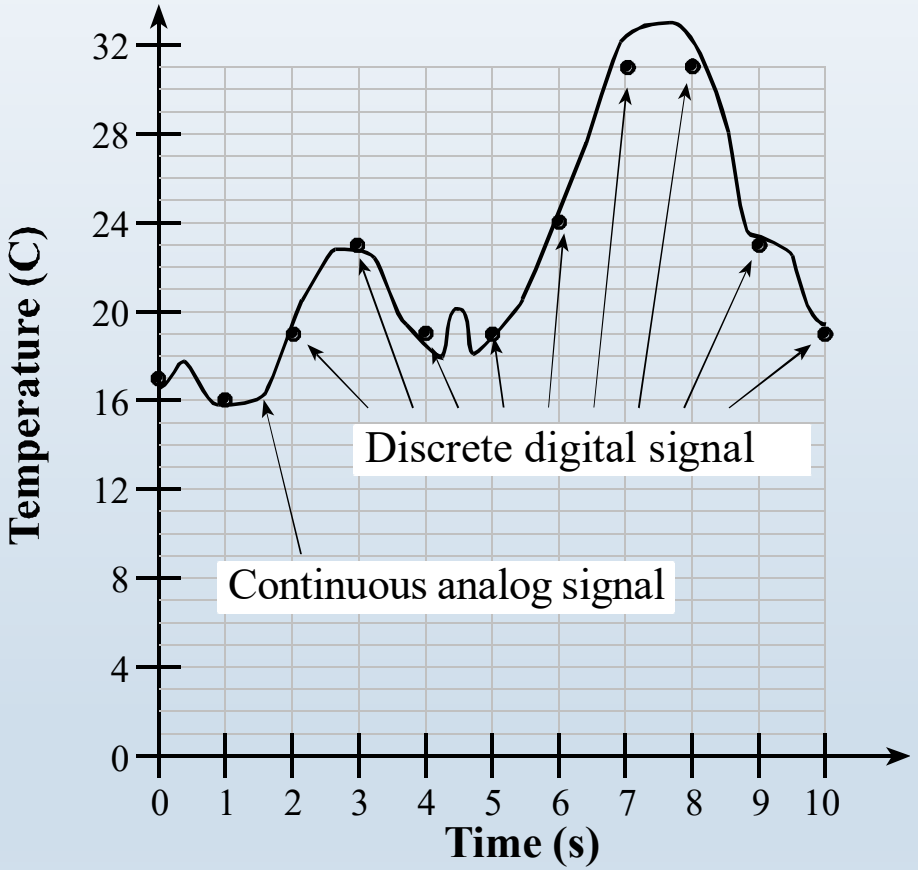
1. Modeling
2. Analysis
3. Design

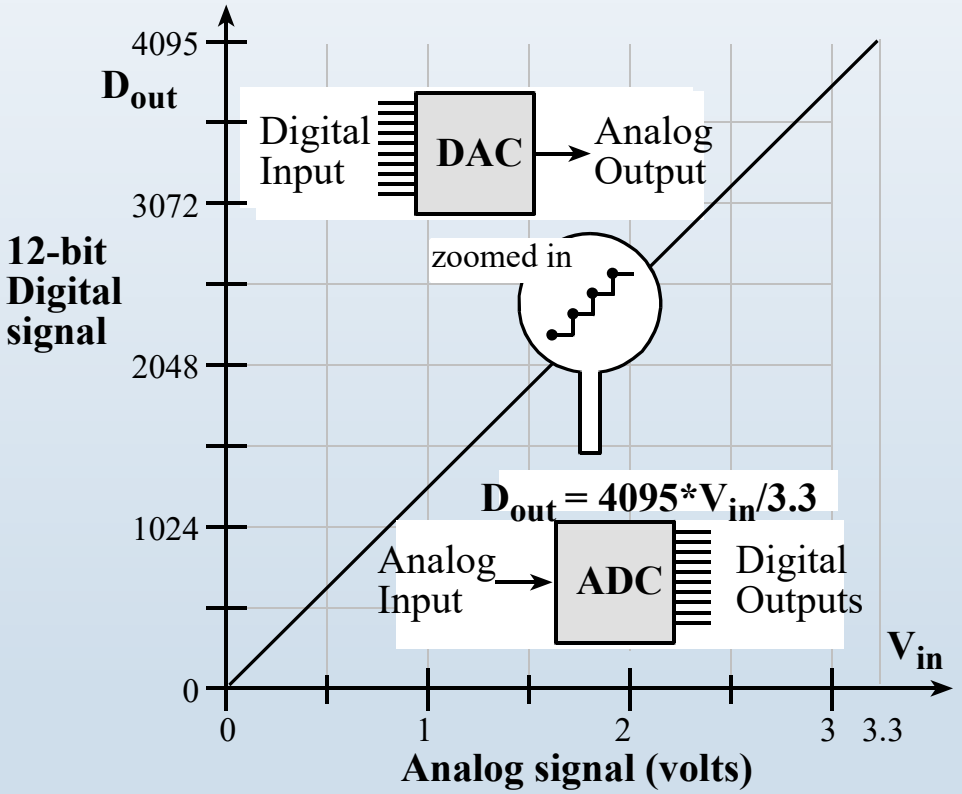


Analog-Digital-Conversion(ADC)?

- Conversion of audio signals (mobile micro, digital music records, ...)
- Conversion of video signals (cameras, frame grabber, ...)
- Measured value acquisition (temperature, pressure, luminance, ...)

A2D Conversion

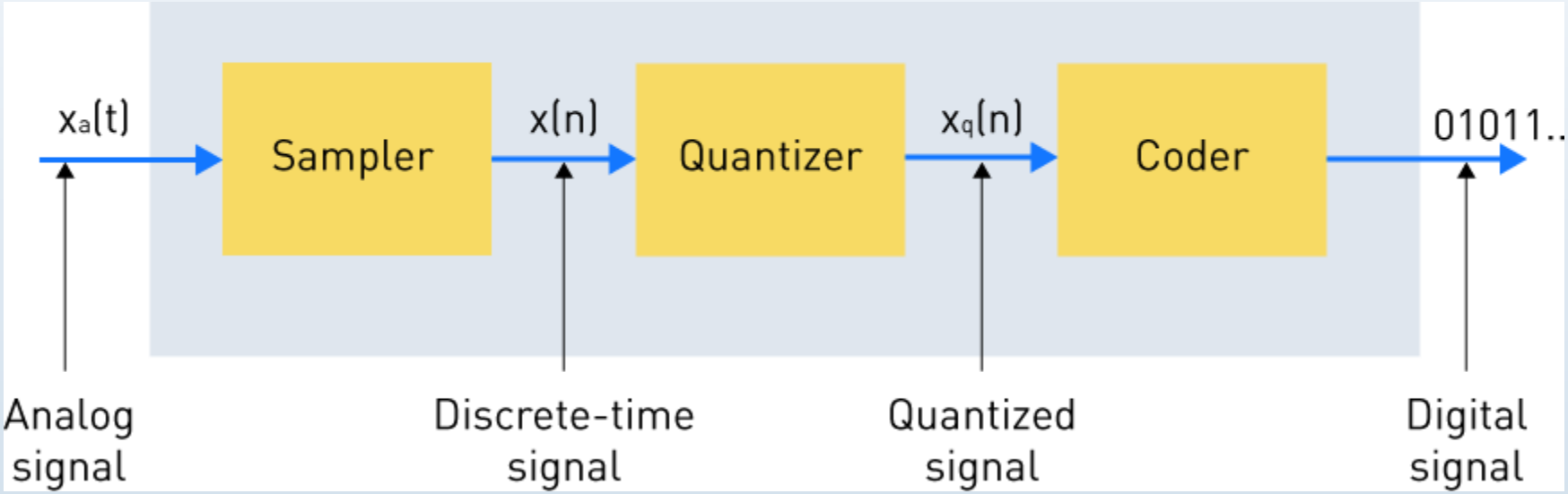




Discrete-time and digital signals

- The discrete-time signal samples may take real (unlimited) values.
- On the other hand, the digital signal samples can only take limited values.
- The discrete-time signal lies between analog (continuous-time) and digital signal representations

A2D Conversion-Scheme



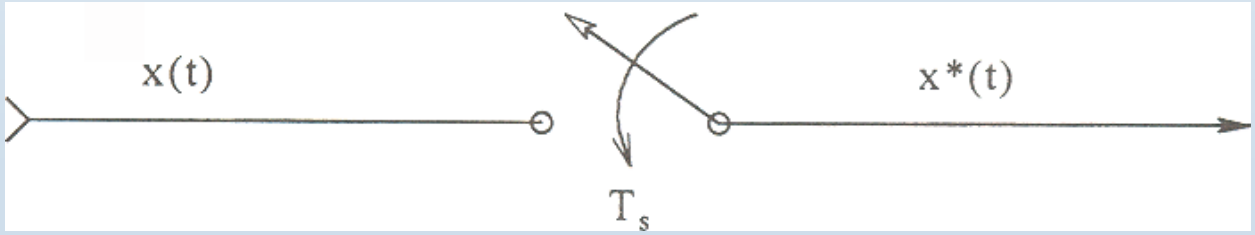
- Analog input can be voltage or current (*in the following only voltage*)
- Analog input can be positive or negative (*in the following only positive*)

A2D- factors

- Sampling
- Quantization
- Coding

Sampling

- **Sampling is the** process of picking one value of a signal to represent the signal for some interval of time.
- Ideally ADC sampling process generates a data sequence from x(t) defined only at the sampling instants Ts,



•Ideal sampling process ADC. T_s—period between successive samples.

Sampling

impulse signal δ(t)

Impulse train: $\delta_T(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT_s)$

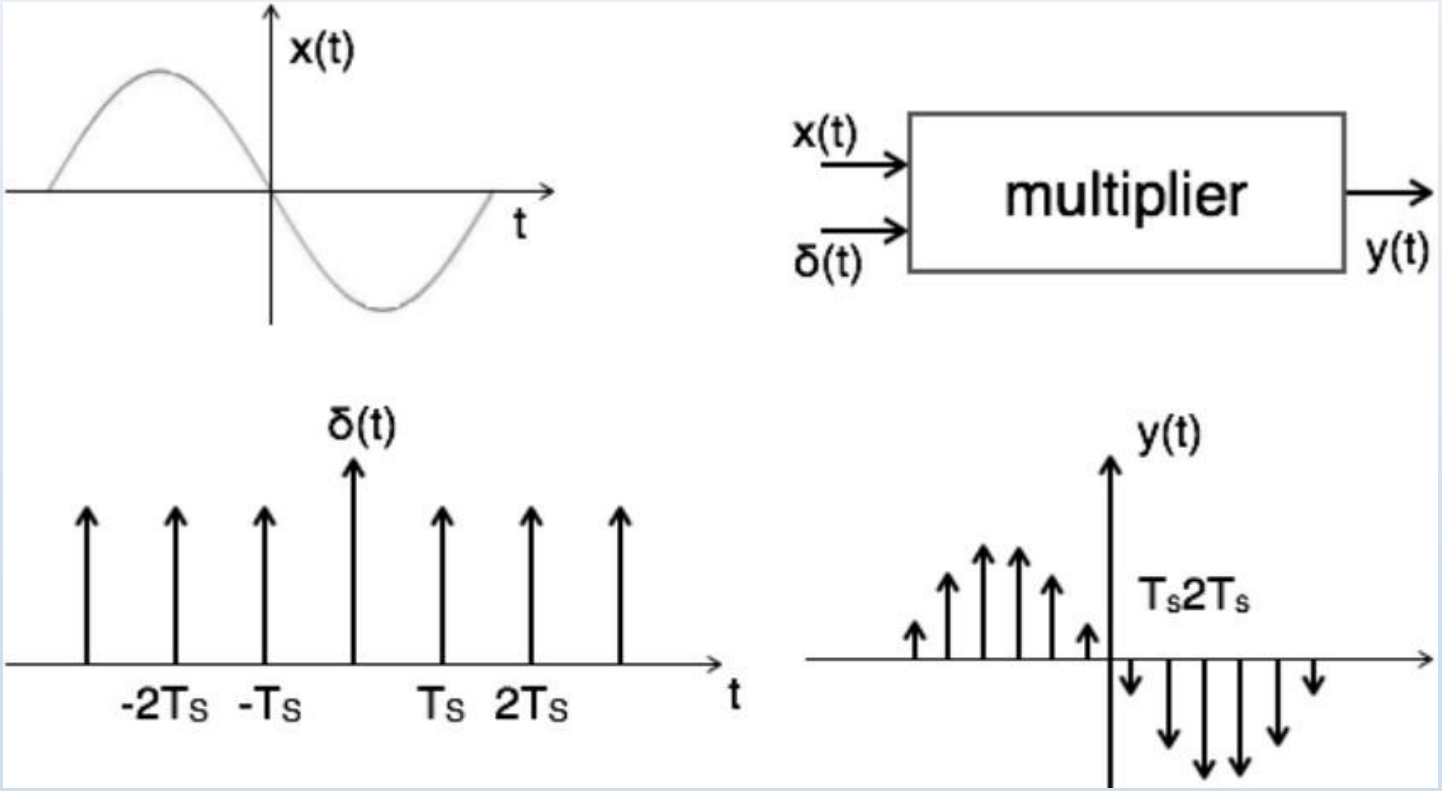
Multiply x(t) with impulse train to get x(nTs) as

$$\sum_{n=-\infty}^{\infty} x(nT_s)\delta(t - nT_s)$$

$\tilde{x}$ [n] = x(nTs)

$\tilde{x}$ [n] represents the discrete-time signal obtained from x(t)

Sampling



Sampling

- What should be Ts? OR fs = 1/Ts?
- Sampling frequency (fs) (or sample rate) is the number of samples per second
- If fs is 44100 hertz, a 60 sec audio will contain 2,646,000 samples
- What will happen if we increase/decrease fs?

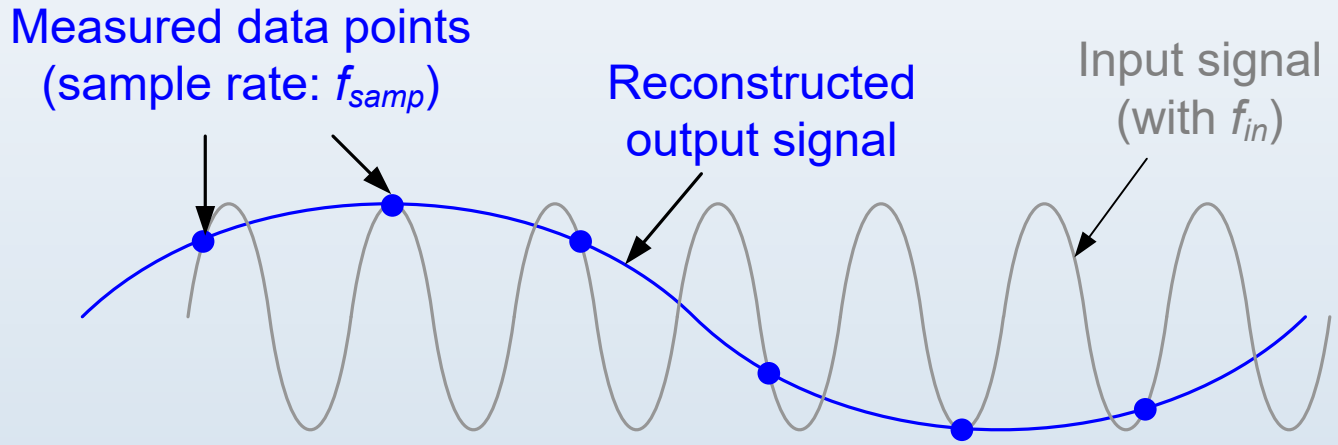
Aliasing

- A bandlimited analog signal that has been sampled can be perfectly reconstructed from an infinite sequence of samples, if the sampling rate

$$fs > 2fmax$$

  - fmax is the highest frequency in the original signal.
  - Fs is the sample frequency
- If the analog signal does contain frequency components larger than (1/2)fs, then there will be an **aliasing error**.
- **Aliasing** is when the digital signal appears to have a
  - **different frequency than the original analog signal**.

Aliasing

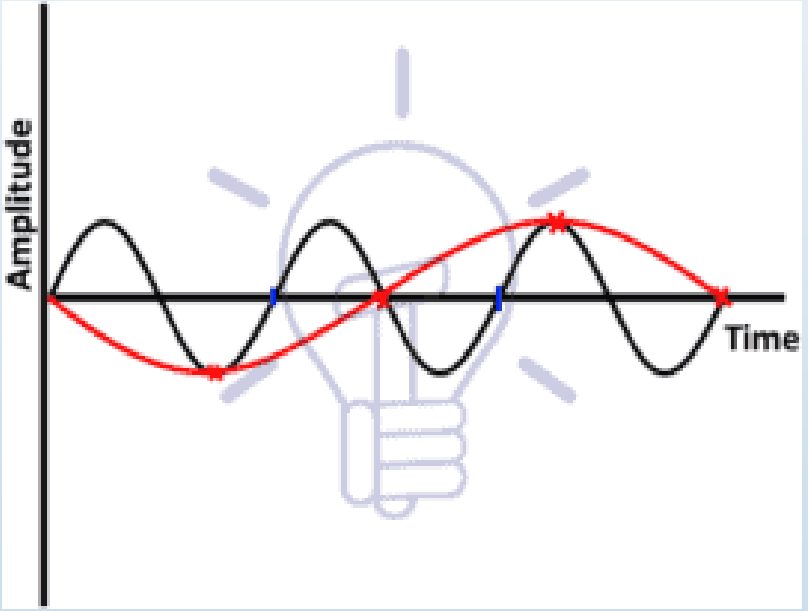


- Too small sampling rate f_{samp} (under-sampling) can lead to aliasing (= frequency of reconstructed signal is to low)

Nyquist criterion for frequency

- f_{samp} more than two times higher than highest frequency component f_{in} of input signal:

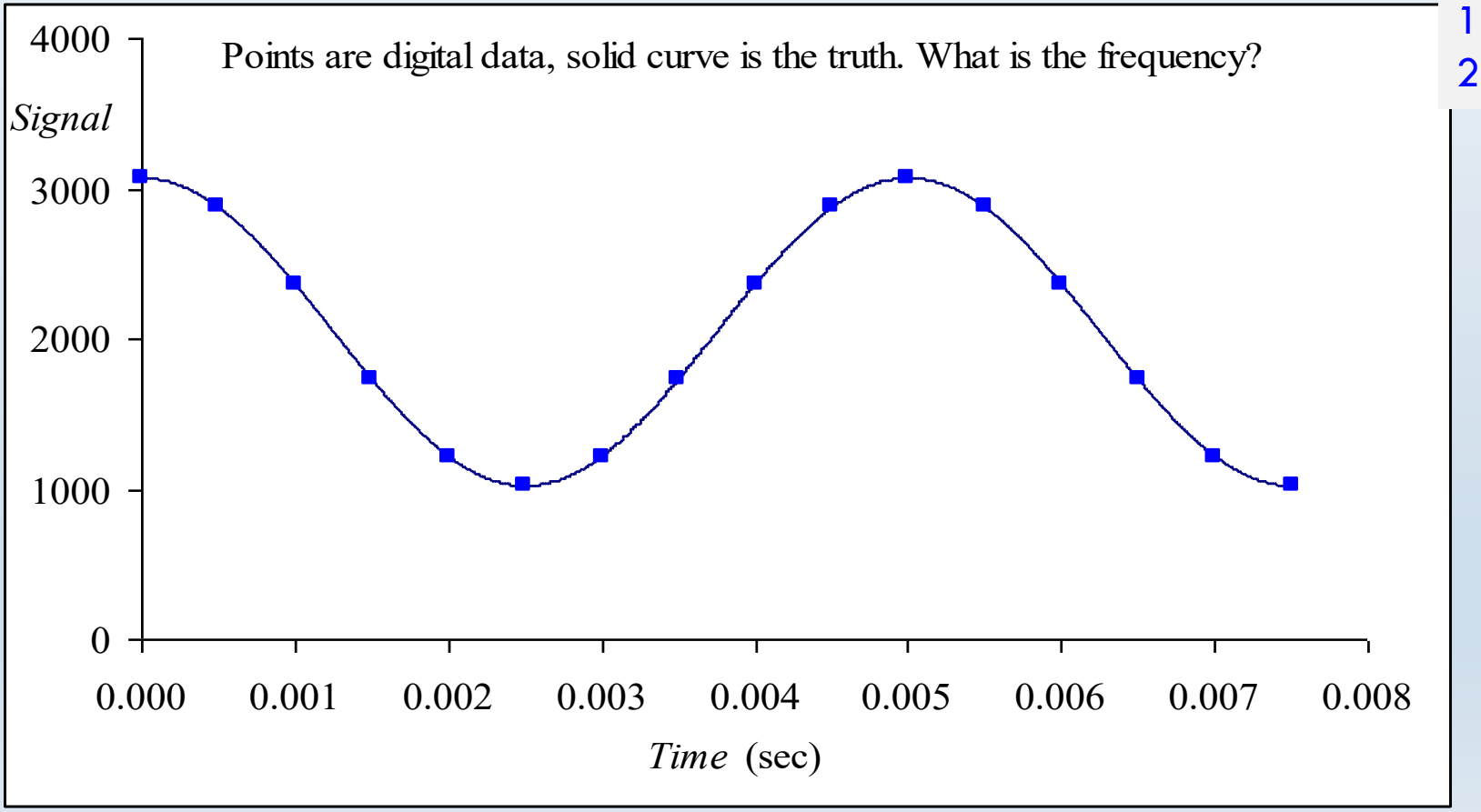
$$f_{samp} > 2 \cdot f_{in}$$



- A signal can be recovered (without loss of information) when the
- $f_{samp} > 2 \cdot f_{in}$

Sampling (option 1)

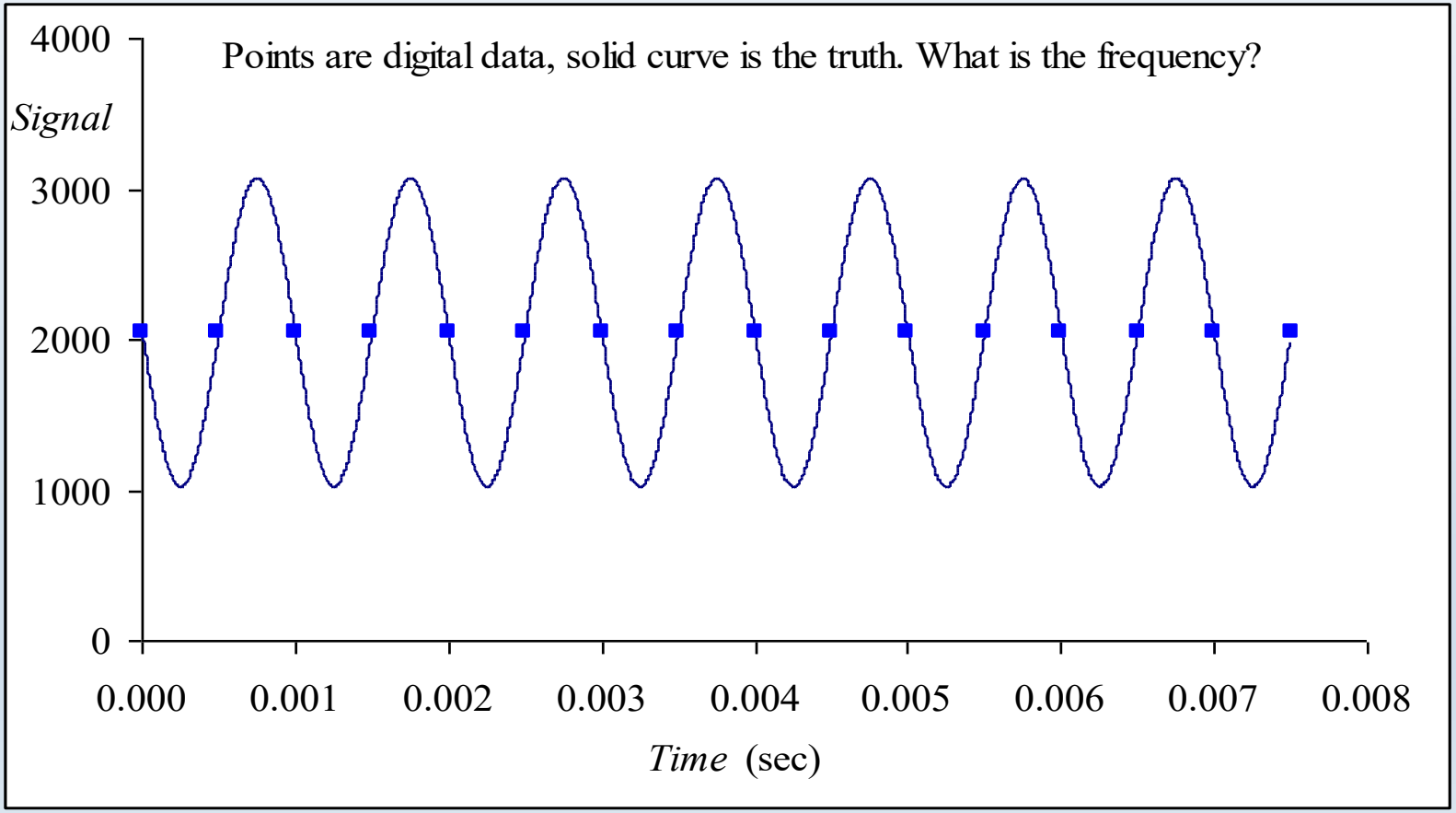
- 200Hz signal sampled at 2000Hz



Look at
200 Hz
1800 Hz
2200 Hz

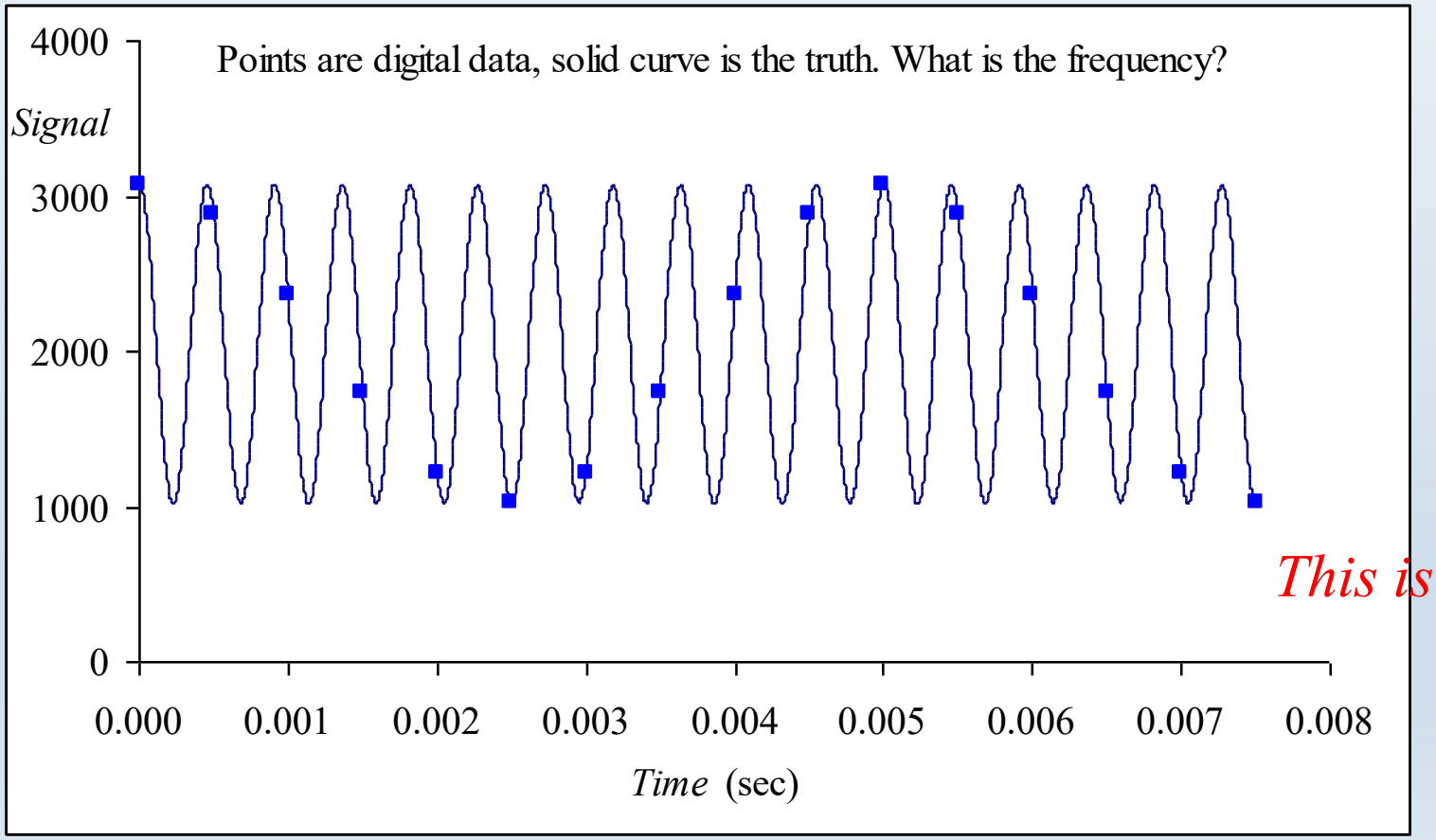
Sampling (option 1)

- 1000Hz signal sampled at 2000Hz



Sampling (option 1)

- 2200Hz signal sampled at 2000Hz



This is aliasing

Quantization

- The process of rounding off the digital signal after the analog signal sample has been converted to digital form.
- Digital signal is represented by a digital (binary) number of a finite number of bits (e.g. 12).
- This limits the **resolution** of the sample (one part in 4096 for 12 bits).

$$x[n_0] = \left\lfloor \frac{(2^N - 1)(\tilde{x}[n_0] - X_{min})}{(X_{max} - X_{min})} \right\rfloor$$

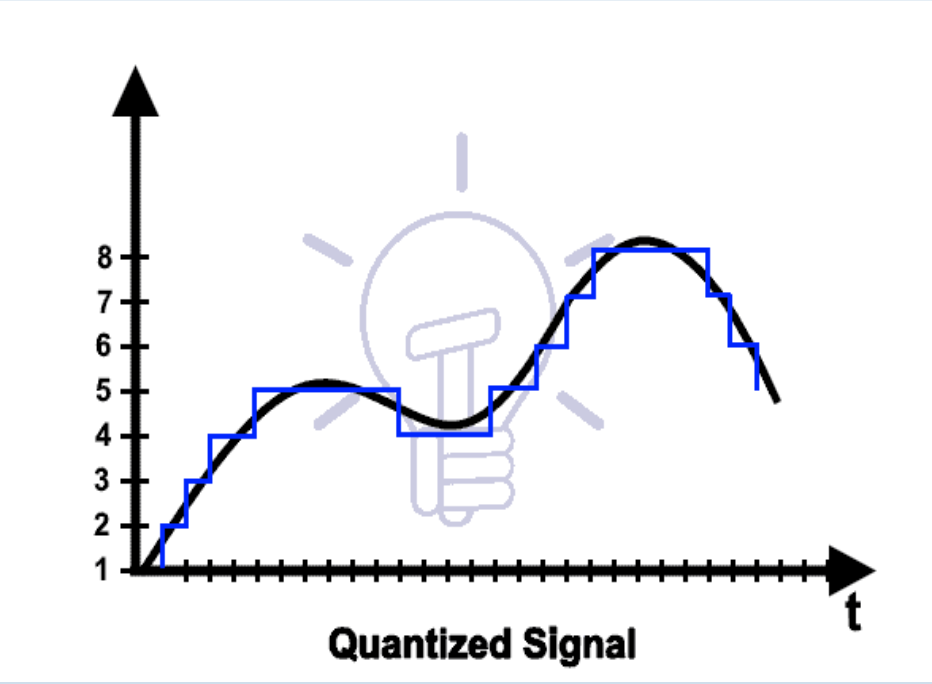
$\lfloor \cdot \rfloor$ stands for the floor function

Steps in Quantization

1. **Defining levels:** The analog input values is divided into a finite number of non-overlapping intervals, or "quantization levels".
2. **Mapping:** Each interval is assigned a single, unique discrete value. The number of levels is determined by the ADC's resolution, expressed in bits.
3. **Approximation:** Then voltage is mapped to the discrete value of the interval it falls into.
4. **Quantization Error:** Since the continuous signal is approximated by these discrete levels, the difference between the true analog value and its assigned digital value is the quantization error.

Exp Quantization

- An analog signal ranging from 0 to +8V.
- Required output is a 3-bit signal
- 3-bit binary number can be used to represent 8 different values, 0 to 7.



Quantization

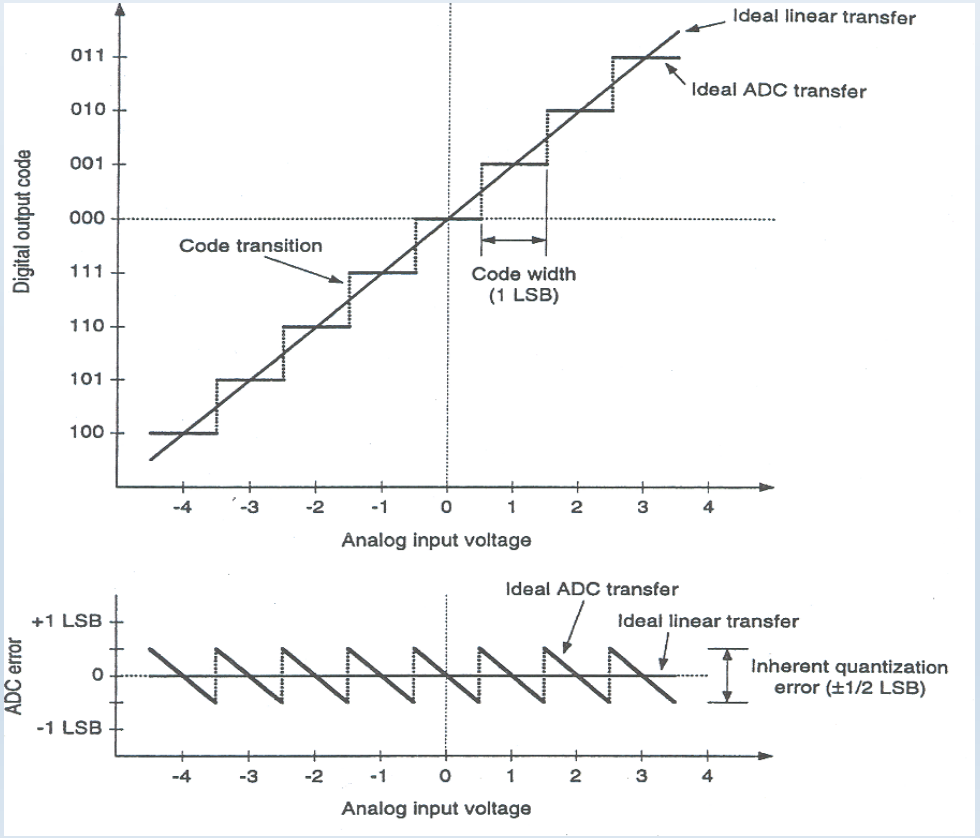
- Resolution of this conversion = 8 V/8 = 1 V.

- If there is a 6.2 V analog level. This falls between 6 and 7, which are multiples of 1 V.
- Since 6.2 is closer to 6, we treat it as if it were 6 V and code it as 110.
- **This process is called quantization.**

- Sampling is done in the time domain and quantization is performed in the amplitude domain.

Quantization

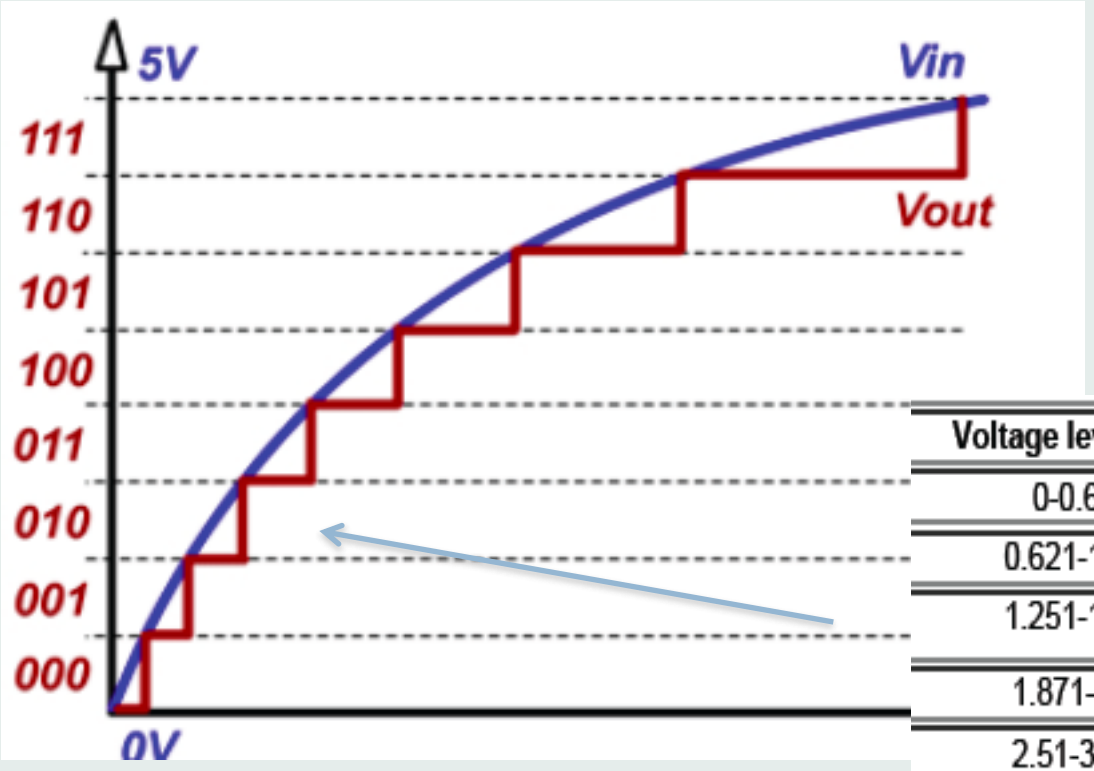
- Normally, an ADC chooses the value closest to the actual sample from a list of uniformly spaced values. This rule gives the transfer function of analog input-to- digital output a uniform “staircase” characteristic.



Coding

- Binary code the quantized values
  Representation of a signal by a binary value in [0,1] form

3-bit conversion example



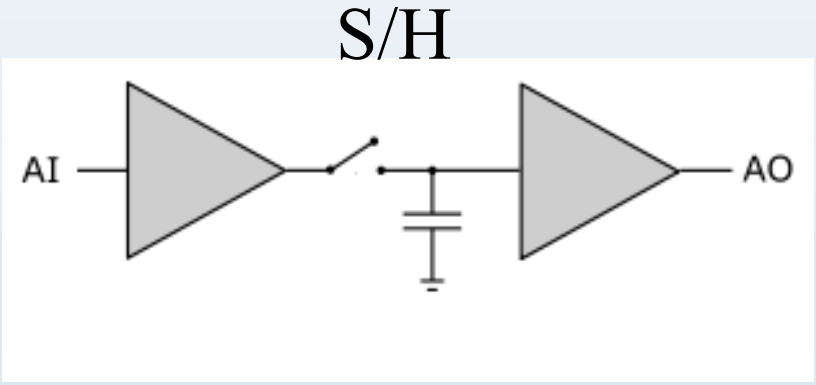
Quantization levels

Voltage levels [V]	Binary representation
0-0.62	000
0.621-1.25	001
1.251-1.87	010
1.871-2.5	011
2.51-3.12	100
3.121-3.75	101
3.751-4.37	110
4.371-5.00	111

Eg 5V divided into 8 levels – each 0.625

Each binary representation is a "range"

Sample-And-Hold Circuit



- Analog Input (AI) is sampled when the switch is closed and its value is *held* on the capacitor where it becomes the Analog Output (AO)

Steps

- Sample and Hold (S&H)
- while conversion takes place voltage must remain stable
- sample voltage – input connected to S&H
- voltage held on a capacitor
- sample time – charging time of capacitor
- input signal disconnected from S&H

Sample and Hold Circuit

Accuracy of A/D Conversion

two ways to improve accuracy:

- Increase resolution: improves accuracy in measuring analog signal amplitude
- Increase sampling rate: increases max frequency that can be measured. Eg high pitch audio

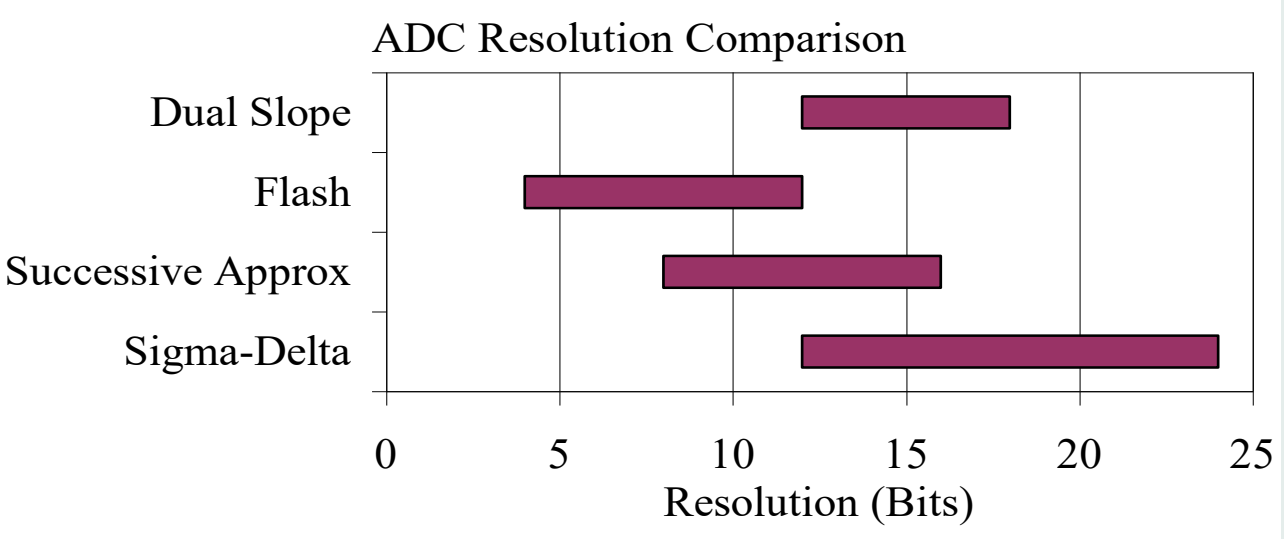
Types of ADCs

- There are 3 major types of ADCs:
  - Flash (parallel) converters,
  - Dual-slope, integrating converters,
  - Successive-approximation converters,

- The fastest ADCs are the flash converters. They can convert 8 bits with a sampling period of less than 1 ns.

- Such fast speed is useful for measuring transient phenomena. E.g. Transient events in particle physics and lasers.

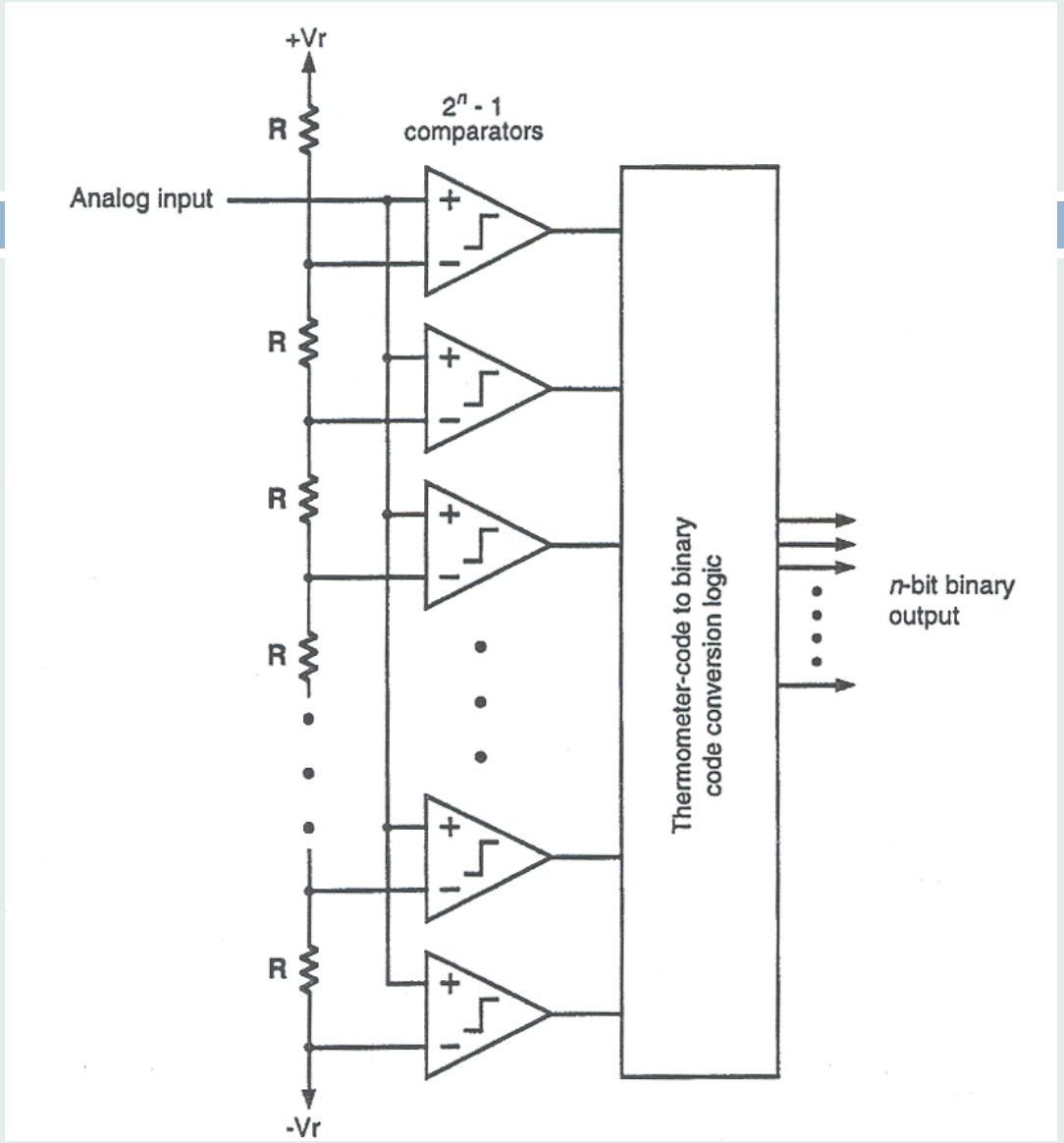
ADC Types Comparison



Type	Speed (relative)	Cost (relative)
Dual Slope	Slow	Med
Flash	Very Fast	High
Successive Appox	Medium – Fast	Low
Sigma-Delta	Slow	Low

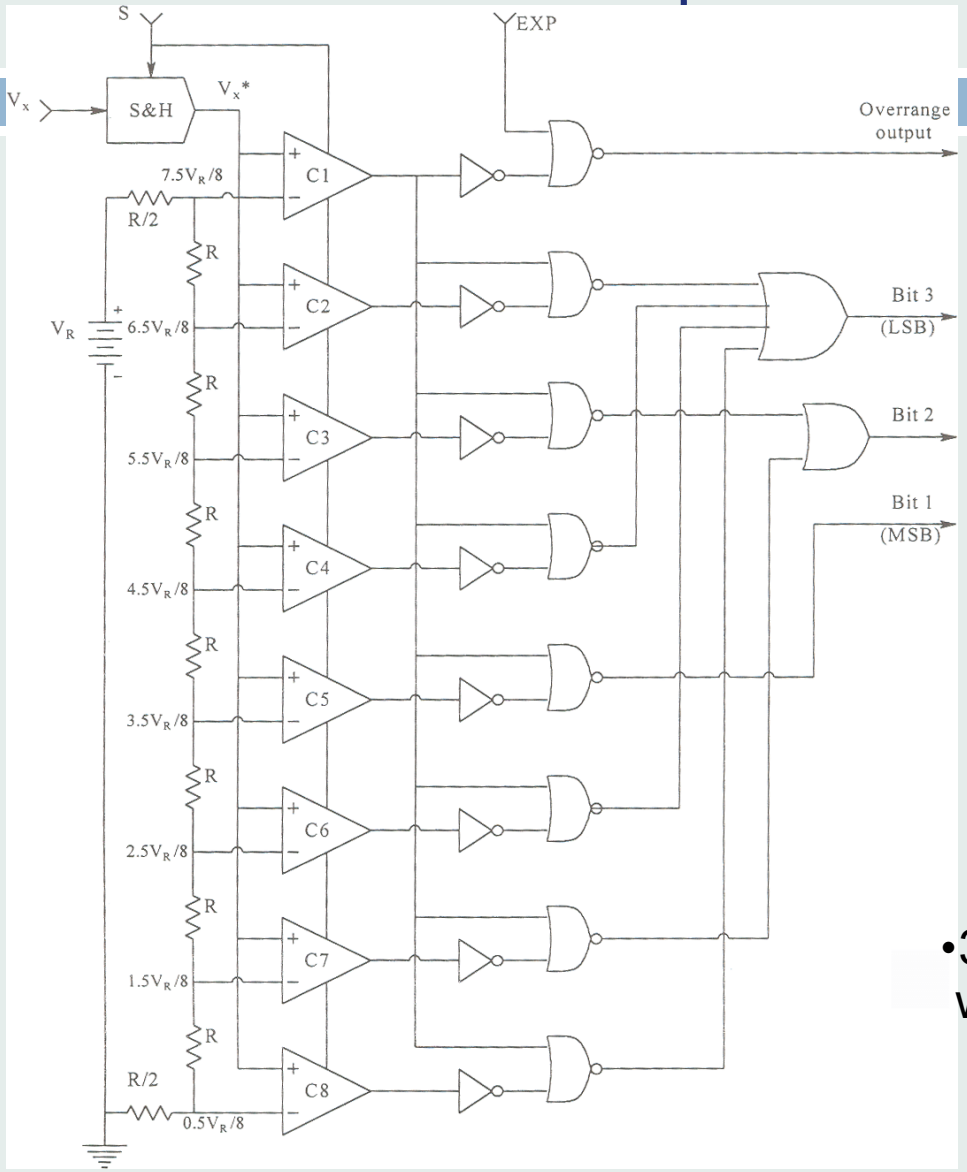
Flash (parallel) converter

- Since **Flash ADCs (FADCs)** is simple-structured, they are fast.
- A string of resistors between two voltage references supplies a set of uniformly spaced voltages that span the input range, one for each comparator.
- The input voltage is compared with all of these voltages simultaneously.
- Comparator outputs = 1 for all voltages below the input voltage
- Comparator outputs = 0 for all the voltages above the input voltage.
- The resulting collection of digital outputs is called a “thermometer code”.



- A flash converter has 2^n-1 comparators operating in parallel.

Flash converter (More Complicated example)



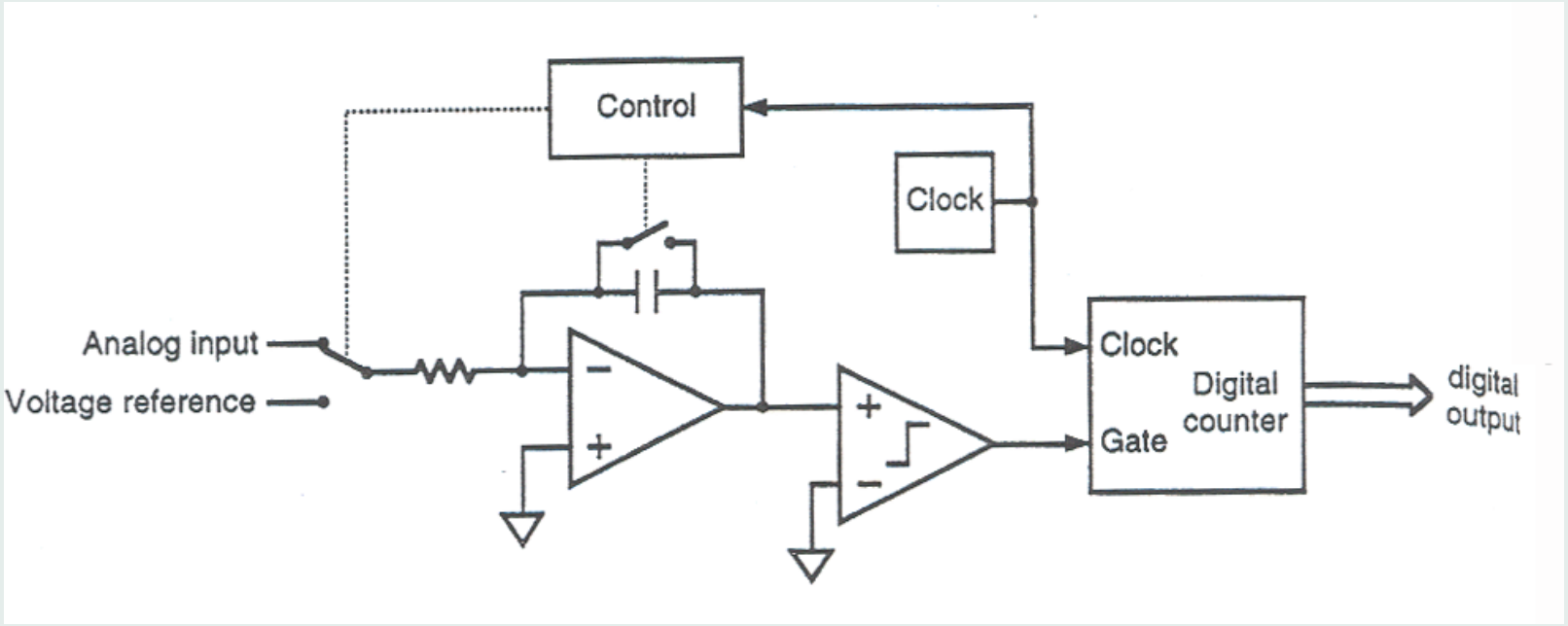
- 3-bit flash ADC with binary output.

Integrating converter

- Integrating converters are used for
  - low- speed,
  - high-resolution

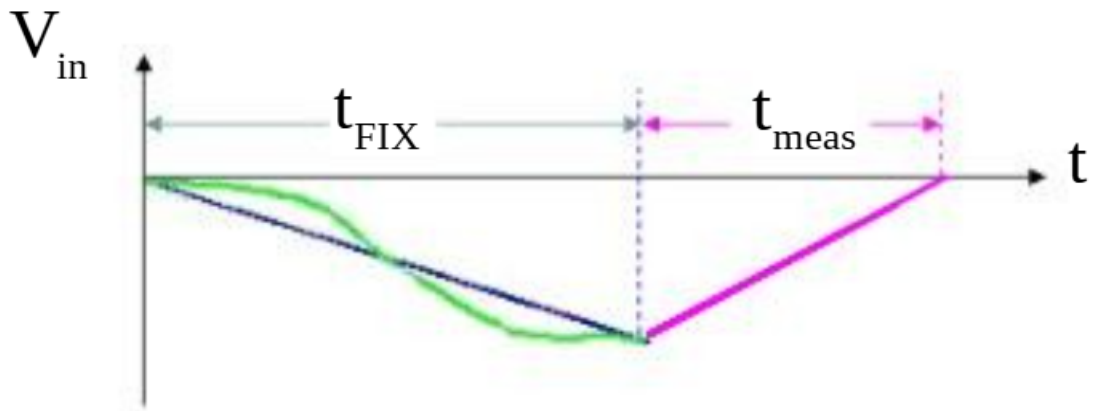
  applications such as voltmeters.
- They are consisting of
  - an integrating amplifier
  - a comparator
  - a digital counter
  - a capacitor for accumulating charge.
- Example: dual-slope ADC

Integrating converter



- A dual-slope integrating converter uses a comparator to determine when the capacitor has fully discharged.

Steps



- The sampled signal charges a capacitor for a fixed amount of time
- By integrating over time, noise integrates out of the conversion
- Then the ADC discharges the capacitor at a fixed rate with the counter counts the ADC's output bits. A longer discharge time results in a higher count

a) Sample

b) hold phase,

c) Charge-redistribution

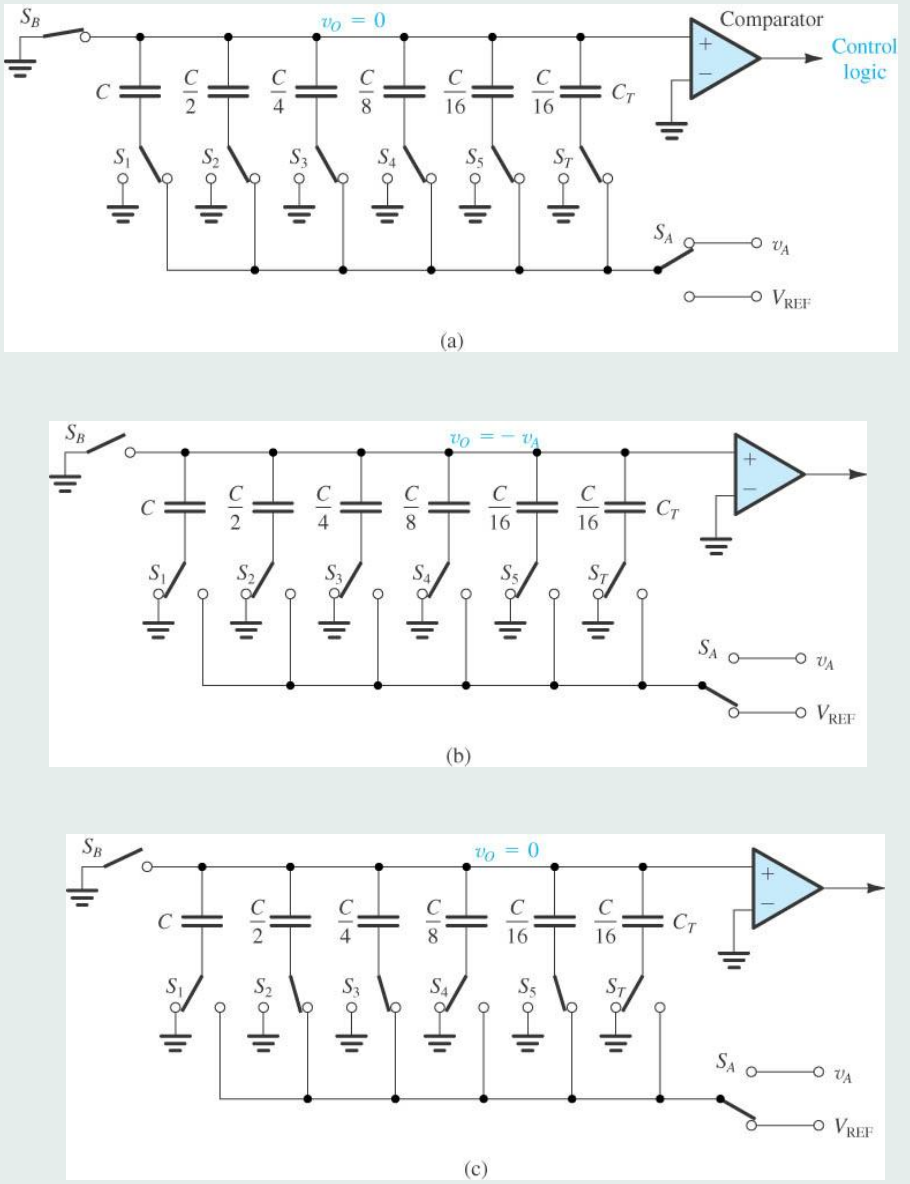
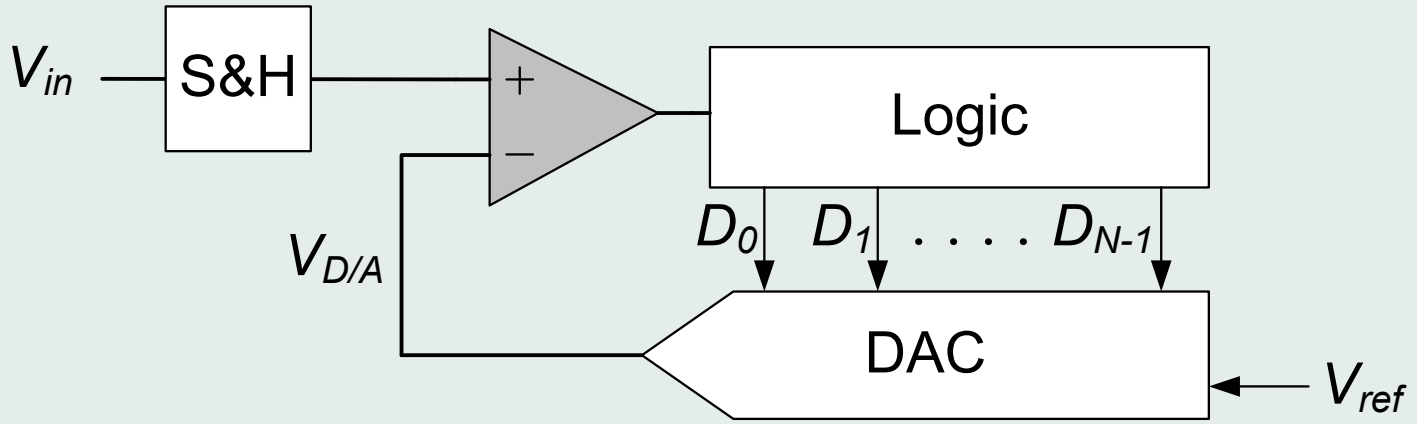


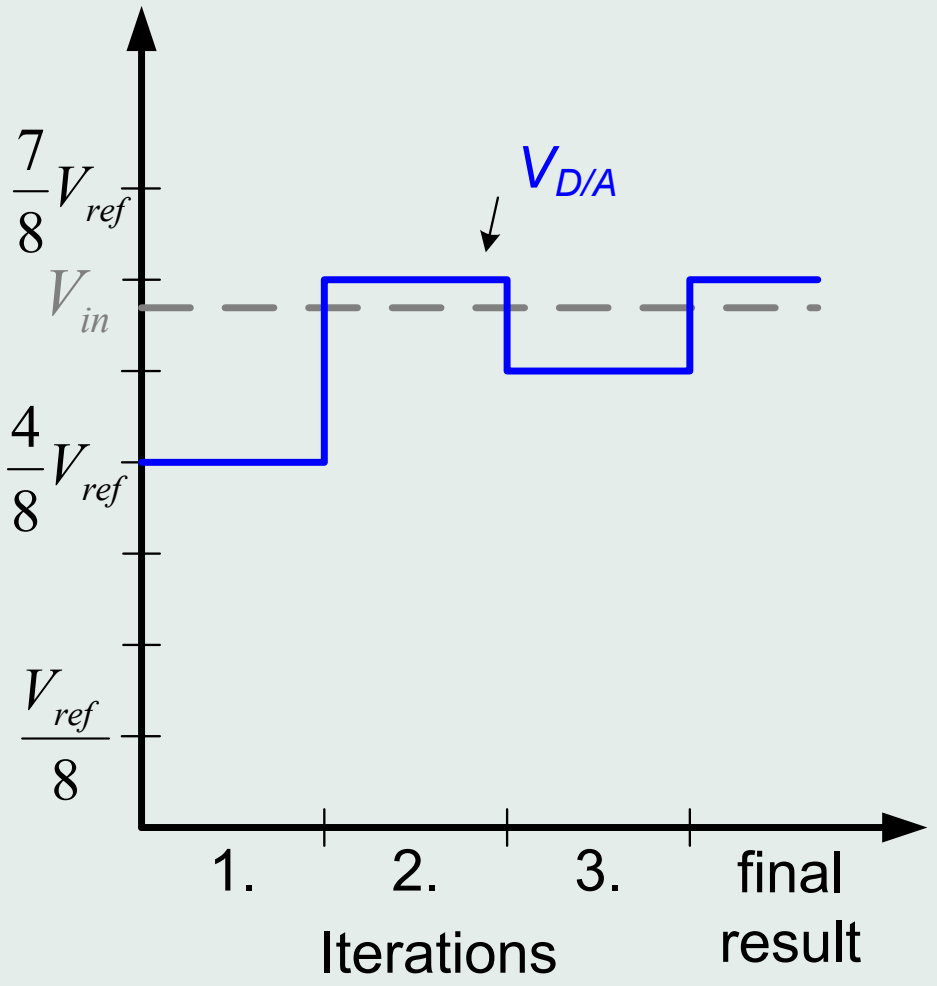
Figure 9.46 Charge-redistribution A/D converter suitable for CMOS implementation: **(a)** sample phase, **(b)** hold phase, and **(c)** charge-redistribution phase.

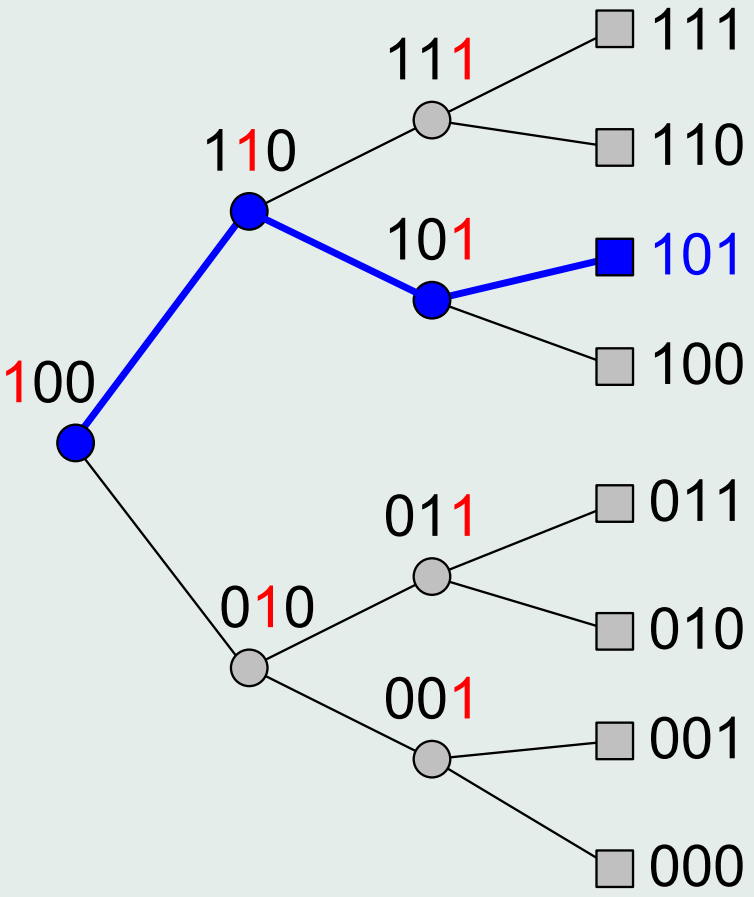
Successive Approximation ADC

- Generate internal analog signal $V_{D/A}$
- Compare $V_{D/A}$ with input signal V_{in}
- Modify $V_{D/A}$ by $D_0D_1D_2...D_{N-1}$ until closest possible value to V_{in} is reached



Successive Approximation ADC cont'd





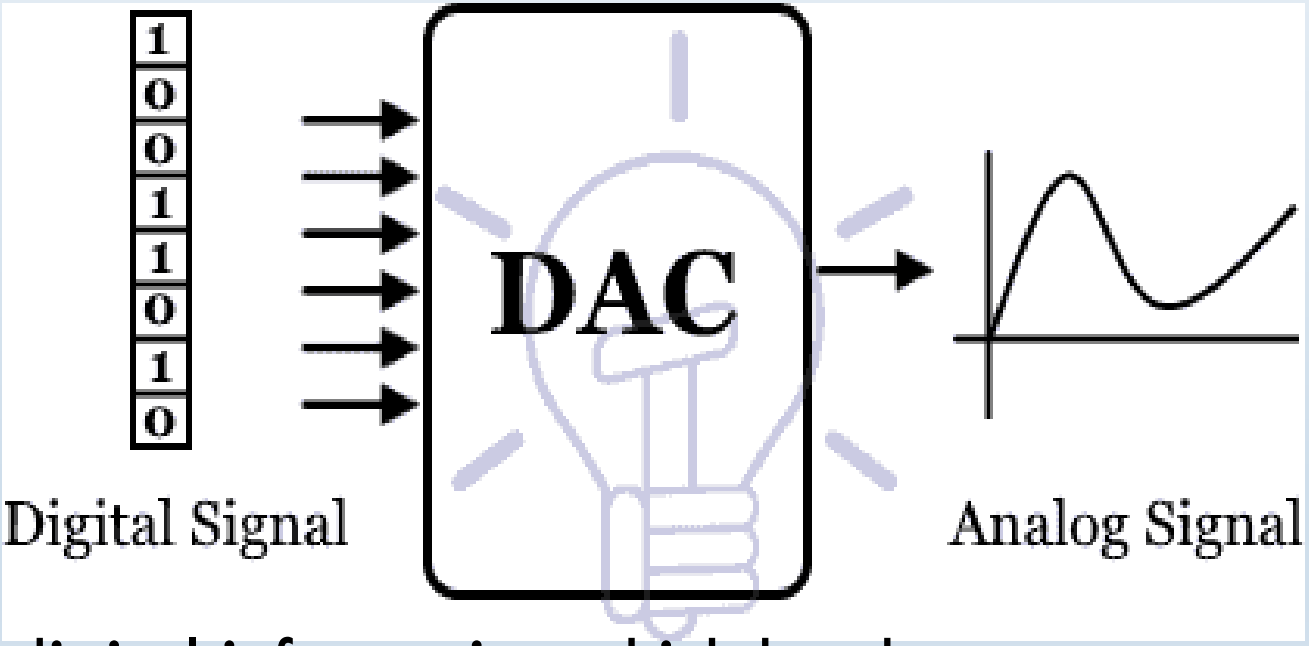
Successive Approx.: pros and cons

- ✓ Low Area / Low Power

- ✘ High effort for DAC
- ✘ Early wrong decision leads to false result

Digital to Analog Conversion

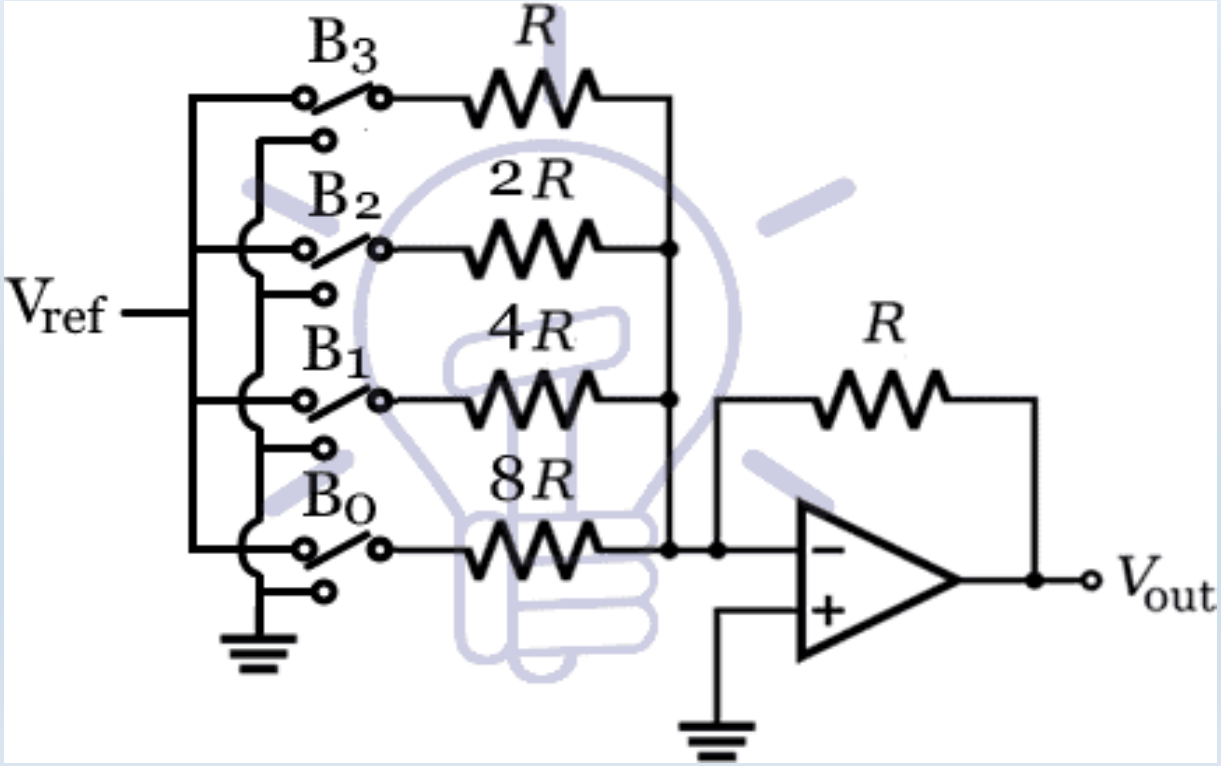
A digital-to-analog converter (DAC) is an integrated circuit (IC) device which converts an N bit digital word to an equivalent analog voltage or current.



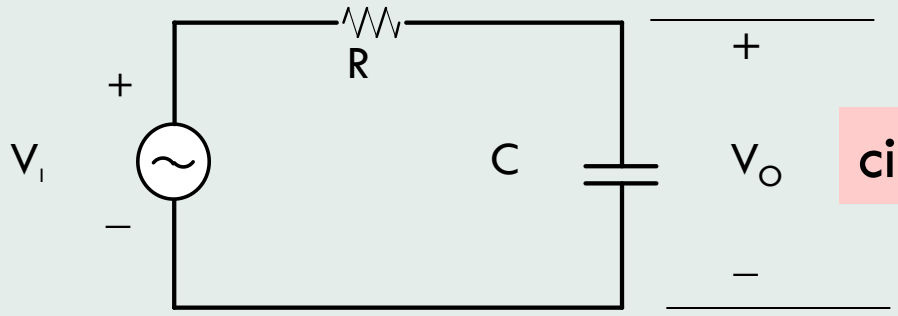
It allows digital information which has been processed and/or stored by a digital computer to be realized in analog form.

Digital to Analog Conversion

A low-pass filter is used to smooth a staircase waveform after digitalization, Called signal reconstruction



Low Pass filter



Low pass filter circuit

circuit of low pass filter.

$$\frac{V_o(jw)}{V_i(jw)} = \frac{\frac{1}{jwC}}{R+\frac{1}{jwC}} = \frac{1}{1+jwRC}$$

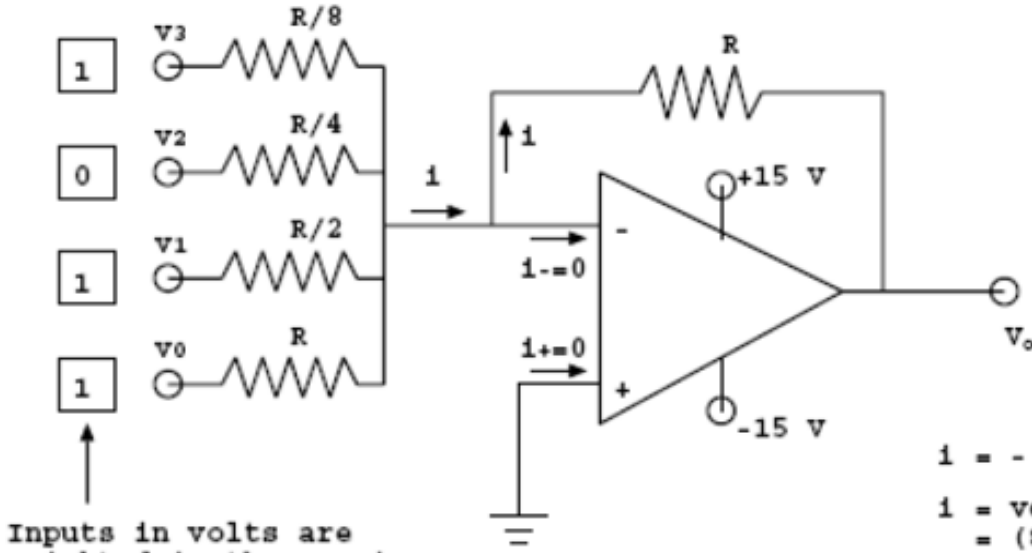
Digital to Analog Conversion

A D/A Converter produces an analog voltage proportional to the digital input.

Example: D/A designed to output 0 to 10 volts.

- Input to D/A is an 8 bit digital value.
- Digital value 0 produces a 0 volt output.
- Digital value 1 produces a 1 x 10/256 volt ouput.
- Digital value 2 produces a 2 x 10/256 volt output.
- Digital value 255 produces a 255 x 10/256 volt output.

Typical 4-bit D/A Converter



Inputs in volts are weighted in the summing amplifier to produce the corresponding analog voltage.

1x8 + 0x4 + 1x2 + 1x1 = 11

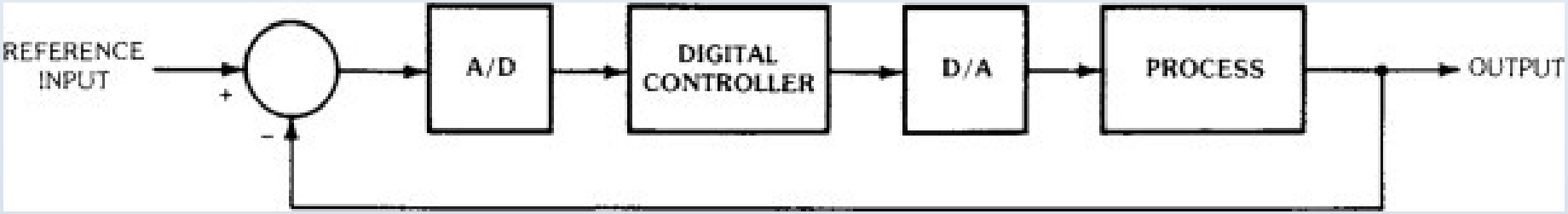
$i = - Vout/R$

$i = v_0/R + 2v_1/R + 4v_2/R + 8v_3/R$
$= (8v_3 + 4v_2 + 2v_1 + v_0)/R$

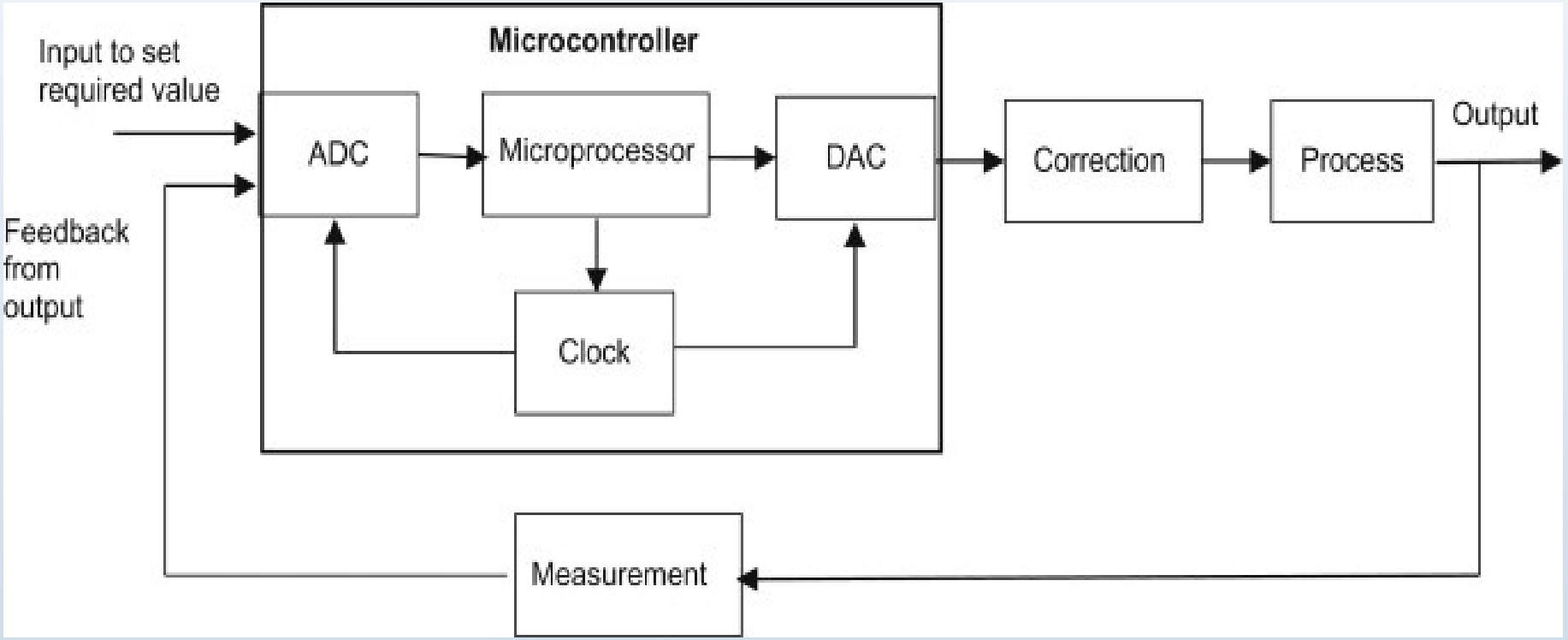
$- Vout/R = (8v_3 + 4v_2 + 2v_1 + v_0)/R$

$Vout = - (8v_3 + 4v_2 + 2v_1 + v_0)$

Control System Block Diagram with A/D



Control System Block Diagram with A/D



ADC in Python.?

Listing 5.5: Representing an analog system in digital form using pole-zero matching method.

```
import control

Ts=0.0005
num=[1000]
den=[1, 1000]

gs = control.TransferFunction(num, den)
gz = control.sample_system(gs, Ts, method='matched')

print('Analog system transfer function',gs)
print('Digital system transfer function',gz)
```

Python Impementation??

Consider the continuous-time system with transfer function

$$G(s) = \frac{20(s+5)}{s(s+4)(s+6)+25}$$

Using the Python control systems library

a) obtain digital version of the system using pole-zero matching,

Set $T_s = 0.05$ seconds

plot unit step response of the obtained discrete systems.

Repeat Problem 5.1 when $T_s = 0.02$ seconds. What has changed?